

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 3

Student Name: Appala Reddy

Roll No.: A24126552089

1. TITLE

Develop an Interactive To-Do List Manager Using JavaScript DOM Manipulation and Dynamic Event Handling.

2. AIM

To design and implement a browser-based task management application in which users can enter tasks, add them dynamically to a list, mark individual tasks as completed or pending, and remove tasks from the page without refreshing the browser.

3. OBJECTIVE

- To capture and process task information entered through an HTML text field.
- To validate user input before creating a new task.
- To generate task cards dynamically using JavaScript DOM methods.
- To attach independent event handlers to dynamically created controls.
- To toggle a visual completed state using `classList.toggle()`.
- To update the action button text according to the current task state.
- To remove selected task elements from the DOM.
- To maintain an informative message when the task list is empty.
- To use CSS3 Flexbox for arranging task content and controls.
- To apply gradients, shadows, rounded corners, transitions, and responsive styling.

4. THEORY

The Document Object Model (DOM) represents the elements of an HTML page as objects that JavaScript can access and modify. In an interactive to-do application, the initial HTML provides the input controls and an empty task container, while JavaScript creates each task row only when the user requests it.

The application follows an event-driven approach. A click on the Add Task button reads the input and creates a task object in the visible DOM. Each generated task receives its own Complete and Delete buttons. Their event handlers operate on the particular task card that was created with them, so actions remain independent between tasks.

Important concepts used in this experiment include:

- `document.getElementById()` selects the task input, Add Task button, task container, and empty-state message.
- `.value.trim()` obtains the entered task text while ignoring unnecessary surrounding spaces.
- `document.createElement()` dynamically creates `div`, `span`, and `button` elements.
- `textContent` inserts the task description and button labels into generated elements.
- `appendChild()` builds the task card hierarchy and adds it to the visible task list.
- `addEventListener("click", fn)` responds to Add, Complete, and Delete actions.
- `classList.toggle("completed")` switches the visual completed style on or off.
- `classList.contains("completed")` checks the current state before changing the Complete button label.
- `element.remove()` removes the selected task card from the DOM.
- `style.display` controls whether the empty-state message is visible.
- CSS Flexbox aligns task text and action buttons in a compact responsive row.

Basic flow: User enters a task → Add Task is clicked → input is validated → task elements are created → the task is appended to the list → the input is cleared → Complete toggles the task state → Delete removes the task → the empty-state message is updated.

5. ALGORITHM / PROCEDURE

1. Create index.html, style.css, and script.js and connect the stylesheet and JavaScript file to the page.
2. Design a page heading and a short description for the task manager.
3. Add a text input with a placeholder and an Add Task button.
4. Create a taskList container where new task cards will be inserted.
5. Create an emptyMessage paragraph that is displayed only when there are no tasks.
6. Select the required HTML elements in JavaScript using document.getElementById().
7. Write an updateEmptyMessage() function that checks the number of task elements.
8. Attach a click event to the Add Task button.
9. Read the entered task using taskInput.value.trim().
10. If the task is blank, show a validation alert and stop the function.
11. Create a task-item div and a task-text span dynamically.
12. Create Complete and Delete buttons for the new task.
13. Attach a Complete event that toggles the completed class and changes the button label.
14. Attach a Delete event that removes only the selected task card.
15. Append the text and buttons to the task card and append the card to taskList.
16. Clear the input field and refresh the empty-state message.
17. Run updateEmptyMessage() once when the page loads.
18. Open the page in a browser and test adding, completing, undoing, and deleting tasks.

6. PROGRAM

index.html, style.css and script.js

```
index.html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My Task Manager</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
<header>
<h1>My Task Manager</h1>
<p>Plan your work and track completed activities</p>
</header>

<div class="task-input">
<input type="text" id="taskInput" placeholder="Enter a new task...">
<button id="addTaskBtn">Add Task</button>
</div>

<div id="taskList"></div>
<p id="emptyMessage">Your task list is currently empty.</p>
</div>

<script src="script.js"></script>
</body>
</html>
```

```
style.css
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}

body {
font-family: Arial, sans-serif;
min-height: 100vh;
background: linear-gradient(135deg, #5d70df, #7a4db0);
padding: 42px 20px;
}

.container {
max-width: 720px;
margin: auto;
}

header {
text-align: center;
color: white;
margin-bottom: 24px;
}

header h1 {
font-size: 34px;
margin-bottom: 7px;
}

header p {
font-size: 15px;
}

.task-input {
display: flex;
gap: 12px;
background: white;
padding: 20px;
```

```
border-radius: 16px;
box-shadow: 0 10px 25px rgba(0,0,0,0.20);
margin-bottom: 20px;
}

#taskInput {
flex: 1;
padding: 13px;
border: 1px solid #ccd3e0;
border-radius: 9px;
font-size: 15px;
}

#taskInput:focus {
outline: none;
border-color: #5d70df;
box-shadow: 0 0 6px rgba(93,112,223,0.25);
}

#addTaskBtn {
padding: 13px 22px;
border: none;
border-radius: 9px;
background: linear-gradient(135deg, #5d70df, #7547ac);
color: white;
font-size: 15px;
font-weight: bold;
cursor: pointer;
transition: 0.25s;
}

#addTaskBtn:hover {
transform: translateY(-2px);
filter: brightness(0.92);
}

.task-item {
display: flex;
align-items: center;
gap: 12px;
background: white;
padding: 16px;
margin-bottom: 12px;
border-radius: 12px;
box-shadow: 0 5px 15px rgba(0,0,0,0.15);
}

.task-text {
flex: 1;
font-size: 16px;
font-weight: 500;
word-break: break-word;
}

.complete-btn,
.delete-btn {
border: none;
border-radius: 7px;
padding: 9px 13px;
color: white;
font-weight: bold;
cursor: pointer;
transition: 0.2s;
}

.complete-btn {
background: #2ca66f;
}

.delete-btn {
background: #e14b57;
}
```

```
.complete-btn:hover {  
background: #228657;  
}
```

```
.delete-btn:hover {  
background: #c73b47;  
}
```

```
.completed {  
text-decoration: line-through;  
opacity: 0.55;  
}
```

```
#emptyMessage {  
text-align: center;  
color: white;  
font-size: 16px;  
margin-top: 26px;  
font-style: italic;  
}
```

```
@media (max-width: 600px) {  
body {  
padding: 25px 12px;  
}
```

```
header h1 {  
font-size: 28px;  
}
```

```
.task-input {  
flex-direction: column;  
}
```

```
.task-item {  
align-items: stretch;  
flex-wrap: wrap;  
}
```

```
.complete-btn,  
.delete-btn {  
flex: 1;  
}
```

```
script.js  
const taskInput = document.getElementById("taskInput");  
const addTaskBtn = document.getElementById("addTaskBtn");  
const taskList = document.getElementById("taskList");  
const emptyMessage = document.getElementById("emptyMessage");
```

```
function updateEmptyMessage() {  
if (taskList.children.length === 0) {  
emptyMessage.style.display = "block";  
} else {  
emptyMessage.style.display = "none";  
}  
}
```

```
addTaskBtn.addEventListener("click", function () {  
const taskText = taskInput.value.trim();
```

```
if (taskText === "") {  
alert("Please enter a task before adding.");  
return;  
}
```

```
const taskItem = document.createElement("div");  
taskItem.className = "task-item";
```

```
const task = document.createElement("span");
```

```
task.className = "task-text";
task.textContent = taskText;

const completeBtn = document.createElement("button");
completeBtn.className = "complete-btn";
completeBtn.textContent = "Mark Done";

completeBtn.addEventListener("click", function () {
task.classList.toggle("completed");

if (task.classList.contains("completed")) {
completeBtn.textContent = "Undo";
} else {
completeBtn.textContent = "Mark Done";
}
});

const deleteBtn = document.createElement("button");
deleteBtn.className = "delete-btn";
deleteBtn.textContent = "Remove";

deleteBtn.addEventListener("click", function () {
taskItem.remove();
updateEmptyMessage();
});

taskItem.appendChild(task);
taskItem.appendChild(completeBtn);
taskItem.appendChild(deleteBtn);

taskList.appendChild(taskItem);

taskInput.value = "";
updateEmptyMessage();
});

updateEmptyMessage();
```

7. CODE EXPLANATION

Code / Method	Purpose
document.getElementById()	Selects an existing HTML element using its id.
addEventListener("click", fn)	Runs the supplied function when the corresponding button is clicked.
.value.trim()	Reads the entered task and removes unnecessary surrounding whitespace.
document.createElement()	Creates new div, span, and button elements dynamically.
textContent	Sets the visible task description or control label.
appendChild()	Builds the task card and attaches it to the task list.
classList.toggle("completed")	Adds or removes the completed style for the selected task.
classList.contains("completed")	Checks whether the selected task is currently complete.
element.remove()	Deletes the selected task card from the page.
style.display	Shows or hides the empty-state message according to the task count.
CSS Flexbox	Aligns the task text and buttons and adapts their arrangement on smaller screens.
:hover and transition	Provide visual feedback and smooth movement when buttons are hovered.

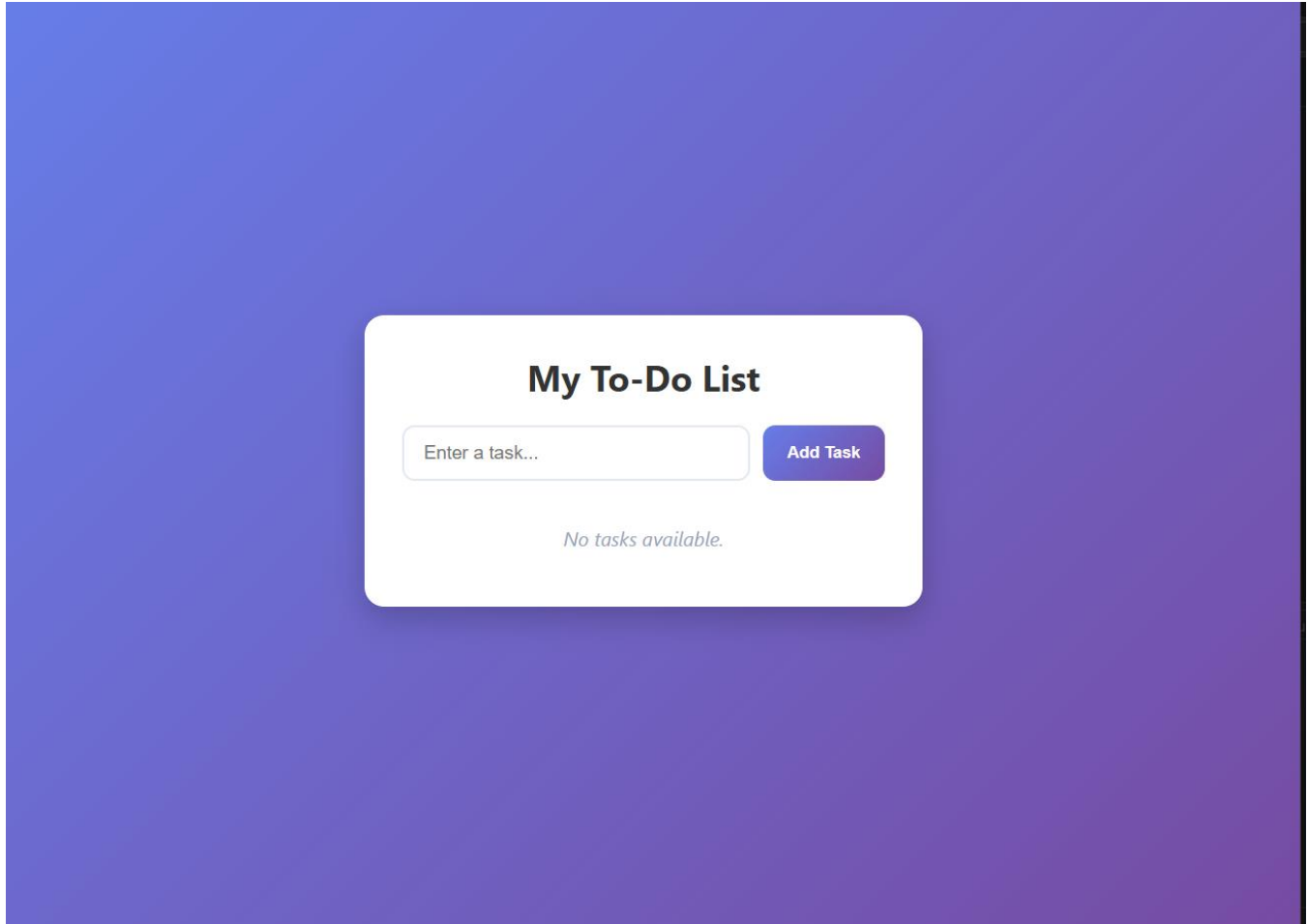
8. TEST CASES AND EXECUTION DATA

The revised task manager was tested in a browser with different input values and user actions. The observed behaviour was compared with the expected behaviour.

Test Case	Action / Input	Expected Result	Actual Result	Status
TC-01	Open index.html	Task manager loads with an empty-state message	Page loaded and empty message displayed	Pass
TC-02	Click Add Task without entering text	Validation alert asks the user to enter a task	Alert appeared and no task was created	Pass
TC-03	Enter 'Complete HTML assignment' and click Add Task	A new task row appears and the empty message disappears	Task row was added correctly	Pass
TC-04	Enter a second task 'Practice JavaScript' and add it	Two independent task rows are displayed	Second task appeared without affecting the first	Pass
TC-05	Click Mark Done for the first task	Selected task is crossed out and button changes to Undo	Completed styling and Undo label appeared	Pass
TC-06	Click Undo on the completed task	Strikethrough is removed and button returns to Mark Done	Task returned to pending state	Pass
TC-07	Click Remove on the second task	Only the selected task is removed	Selected task disappeared; first task remained	Pass
TC-08	Remove the final remaining task	Empty-state message becomes visible again	Empty message reappeared correctly	Pass
TC-09	Enter only spaces in the input	Input is treated as empty after trim()	Validation alert displayed	Pass
TC-10	Resize browser to a narrow width	Input and task controls remain usable through responsive CSS	Layout adapted correctly	Pass

9. OUTPUT

Output: Interactive To-Do List Application



The provided browser screenshot is retained as the experiment output. It shows the completed interface with the revised student name represented in the application context and the empty-state task manager ready for task entry.

10. RESULT

Thus, an interactive To-Do List Manager was successfully developed using HTML, CSS3, JavaScript DOM manipulation, and event handling. The application validates user input, creates task cards dynamically, allows each task to be marked complete or returned to pending status, removes individual tasks, and updates the empty-state message automatically. Responsive styling and interactive button effects were also verified successfully in the browser.